Физика: твёрдые тела, SDF-коллизии и разрушение

Статус: текущая alpha-ветка. Это руководство по физическому слою, который игра использует для движущихся тел, SDF-рельефа, разрушаемых объектов и нового контракта «удар — материал — урон». Это инженерное руководство, а не обещание, что движок берёт на себя все игровые правила. Пороги урона, лут, условия победы и поражения, арт фрактур и партиклы остаются на стороне игры.

0. Выбор бэкенда, возможности и снаряды

В движке два бэкенда под одним API `PhysicsSystem`:

Бэкенд	Для чего	Соединения	CCD	Детерминизм	Рекомендуемое число тел
`PhysicsBackendKind::Box2D`	обычные твёрдые тела, верёвки, машины, рэгдоллы, редактор	да	да	зависит от платформы	примерно до 10000
`PhysicsBackendKind::Dod`	большие стратегические миры без соединений, простые формы, снаряды	нет	только swept-запросы	фиксированный порядок SoA	растёт вместе с числом живых тел
`PhysicsBackendKind::Auto`	политика по умолчанию	выбирает Box2D при необходимости соединений	выбирает Box2D при запросе CCD	выбирает DOD при детерминизме или бюджете 4096+	зависит от выбранного бэкенда

`Auto` — значение по умолчанию. Задайте `PhysicsDesc::backend` до `PhysicsSystem::Initialize`, вызовите `SelectBackend` до создания мира или используйте `SetProjectPolicy`, чтобы обновить политику для Play-In-Editor.

Живой мир никогда не переключается на лету.

Настройка проекта

Проект хранит:

```
[physics]
backend = 0      # 0 Auto, 1 Box2D, 2 DOD
body_budget = 0  # Auto выбирает DOD при 4096 и больше
```

Те же значения редактируются в **Project Settings > Physics** и применяются к следующему Play-In-Editor или запуску игры; эталонный мир редактора сохраняет собственный бэкенд.

Возможности

`PhysicsSystem::Capabilities()` сообщает наличие соединений, CCD, детерминизма, больших миров и рекомендуемый потолок тел. Не угадывайте поддержку по имени бэкенда — запрашивайте возможности. Запрос соединения в DOD

даёт явную ошибку; переход на Box2D происходит только в режиме `Auto`.

Часы тела, gravity scale и контакты

`TimeScaleComponent` замораживает тело при нулевом или отрицательном `Scale`: оба бэкенда сбрасывают скорость, угловую скорость и накопленное ускорение, а масштаб в (0, 1) масштабирует авторские импульсы, ускорение и интегрирование. `RigidBody2D::GravityScale` умножает мировую гравитацию на тело. Контакты сообщают `BeginTouch` один раз при начале пересечения пары и `EndTouch` один раз при её расхождении; сенсоры дают те же события пары, не разрешая контакт.

Режимы снарядов

`Projectile2D` — ECS-компонент:

Поле	Значение
`Mode`	`kProjectileModeAuto`, `Hitscan`, `Swept`, `Dynamic`
`Speed`	пикселей в секунду; ноль означает скорость тела
`Radius`	радиус для выбора Auto
`MinWallThickness`	минимальная гарантируемая толщина стены
`Direction`	нормализованное направление
`MaxSubsteps`	резерв для динамической интеграции
`CollisionMask`	маска категорий для segment query

`Auto` сравнивает `Speed * fixedDt` с `MinWallThickness * 0.5`. Малый радиус даёт `Hitscan`, крупный/физический снаряд — `Swept`, медленный остаётся `Dynamic` и получает bullet/CCD-флаг Box2D. Режим всегда можно задать вручную, поэтому разработчик, знающий толщину стены и скорость, не зависит от эвристики.

`ProjectileSystem::Step` выполняется до следующего шага физики для hitscan/swept и записывает `ProjectileHitEvent` с целью, точкой, нормалью и скоростью. Решение об уроне, эффекте и удалении снаряда остаётся за игрой.

Скрипты и визуальный язык

В текстовом FemboyScript есть `physics_get_backend()`, `physics_set_backend(backend)` и `physics_get_capabilities()`. В палитре LogicGraph есть узлы `Physics Get Backend`, `Physics Set Backend` и `Physics Capabilities`, которые компилируются в те же natives и AOT-вызовы. Старые скрипты без упоминания бэкенда используют `Auto`.

Тесты и бенчмарки

- `physics_backend` — Auto-выбор, возможности, segment queries и защита от туннелирования снарядов.
- `script_physics` — текстовый и байткод-мост.
- `physics_backend_benchmark` — ms/frame для Box2D и DOD при настраиваемом числе тел (по умолчанию 2000, можно передать аргументом).

1. Единицы, координаты и порядок шага

- Мировые позиции задаются в пикселях. Box2D работает в метрах; `PhysicsDesc::pixelsPerMeter` — единственная точка пересчёта.
- `Transform2D::Position` — авторитетная геймплейная позиция. `RigidBody2D` хранит упакованный id Box2D и последнюю интерполированную позу.
- Симуляция выполняется в `Phase::FixedUpdate` с фиксированным `dt`. Рендер интерполирует между `PreviousPosition` и `Position`.
- Рекомендуемый порядок: ввод, движение, `PhysicsSystem::Step`, `SdfCollisionSystem::Step`, обработка ударов и урона, фрактура, затем интерполяция рендера.
- Тело, уничтоженное между шагами, не должно оставлять упакованный id формы;

`PhysicsSystem::DestroyAllBodies` и очистка реестра отвечают за teardown.

2. Твёрдые тела и соединения

`RigidBody2D` и `Collider2D` — ECS-компоненты. Это обычные данные, в них нет `box2d.h`. `PhysicsSystem` владеет миром; системы соединений, транспорта и колёс — явно знающие Box2D единицы трансляции за теми же упакованными handles.

```
Fby::PhysicsSystem Physics;
Fby::PhysicsDesc Desc;
Desc.gravity = Fby::Vector2D{0.0f, 980.0f};
Physics.Initialize(Desc);

const Fby::Entity Box = Registry.create();
Fby::Transform2D Transform;
Transform.Position = Fby::Vector2D{120.0f, 80.0f};
Registry.emplace<Fby::Transform2D>(Box, Transform);

Fby::RigidBody2D Body;
Body.Type = Fby::BodyType::Dynamic;
Registry.emplace<Fby::RigidBody2D>(Box, Body);

Fby::Collider2D Collider;
Collider.HalfExtents = Fby::Vector2D{16.0f, 16.0f};
Registry.emplace<Fby::Collider2D>(Box, Collider);

Physics.EnsureBodies(Registry);
```

Соединения (`PhysicsJoints.hpp`) реализуют цепь, верёвку и пружину в фиксированном графе. Ткань и мелкая вода — отдельные системы; они не проходят через решатель твёрдых тел.

3. SDF-рельеф

`SdfField` — невладеющее представление плотной сетки расстояний. Отрицательные значения означают материал, положительные — внешнюю область.

`SdfField::Sample` выполняет билинейную выборку, `SdfField::Gradient` возвращает внешнюю нормаль.

`SdfBrickMap` хранит то же поле блоками 16x16: `Empty`, `Full` или `Narrow`.

Карта блоков — это broadphase и GPU-зеркало, а не второе физическое представление.

`SdfCollisionSystem` опрашивает центр и углы коллайдера, находит самое глубокое проникновение, берёт нормаль из градиента и корректирует тело скоростью и импульсом. Коррекция выполняется через скорость, а не телепорт трансформа, поэтому непрерывность контакта и CCD сохраняются.

3.1 Данные контакта

Каждый разрешённый контакт — это `SdfContact`:

Поле	Значение
`entity`	Сущность тела
`point`	Самая глубокая точка контакта в мировых пикселях
`normal`	Внешняя нормаль поверхности
`penetration`	Глубина проникновения
`impulse`	Величина корректирующего импульса
`RelativeSpeed`	Скорость сближения по нормали до коррекции
`KineticEnergy`	`0.5 * Mass * RelativeSpeed^2`
`Mass`	Масса тела на момент контакта
`sequence`	Монотонный номер контакта

`RelativeSpeed`, `KineticEnergy` и `Mass` фиксируются **до** выталкивания тела. Правило урона должно использовать эти поля, а не скорость выталкивания после коррекции, иначе один и тот же удар даст разный урон в зависимости от бюджета коррекции.

`SdfCollisionSystem::Contacts()` возвращает не более `kMaxContacts` контактов за шаг. `SdfCollisionSystem::Manifolds()` возвращает до четырёх точек на тело для отладки и будущей работы решателя.

3.2 Ограничения SDF-контактов

- SDF-контакты описывают только тело против поля. Контакты тело-тело по-прежнему

приходят из Box2D; для урона между двумя динамическими телами используйте скорость и импульс контакта Box2D.

- Корректирующий импульс не физически точный контактный импульс.

Честный вход для правила урона — новое поле `KineticEnergy`.

- Тонкие элементы исчезают на разрешении поля. Разрез тоньше примерно двух

ячеек не надёжная геймплейная граница.

- Градиент не определён на медиальной оси. Система коллизий использует предыдущую нормаль вместо выдуманного направления.

4. Геометрия разрушения

`SdfDestructible` — локальное плотное поле, принадлежащее одному объекту.

- `Configure` выделяет локальную сетку по `SdfDestructibleDesc`.
- `Subtract(SdfCarveOp)` добавляет операцию вычитания и перепекает только затронутые блоки.
- Список операций — источник истины. `SaveOps` сохраняет список, `LoadOps` воспроизводит его; испечённые сэмплы — кэш.
- `kMaxOps` равно 64. Если игре нужно больше деталей, следует использовать заранее нарезанные фрагменты, а не неограниченный список вырезаний.
- `SdfWorld::Imprint` впечатывает поле осевшего объекта в мир. До этого момента мировое поле объект не видит.

`SdfFracture.hpp` режет разрушаемый объект на сетку кусков для прототипа.

`SdfDebris.hpp` детерминированно создаёт прямоугольные осколки из материала, который удаляет вырезание. Оба варианта простые; для production-разрушения нужны авторские фрагменты со своими спрайтами, выпуклыми формами, массами и материалами.

`SdfDestructionCommands.hpp` записывает операцию вырезания в командный лог. Реплей, получивший тот же список операций, восстановит ту же дыру.

5. Инфраструктура удара, материала и урона

Новый заголовок — `Engine/Physics/SdfDamage.hpp`. Он использует фиксированные ёмкости, детерминирован, не выделяет память и не зависит от рендера.

5.1 Профиль материала

`SdfMaterialProfile` — это данные, а не поведение:

Поле	По умолчанию	Значение
`Toughness`	1.0	Сопротивление урону
`Hardness`	1.0	Множитель урона при том же

		ударе
`MinKineticEnergy`	0.0	Энергия, ниже которой удар игнорируется
`EnergyToDamage`	1.0	Урон на единицу эффективной энергии
`ThicknessExponent`	1.0	Насколько локальная толщина снижает урон
`MaximumDamageFraction`	1.0	Максимальная доля целостности, снимаемая одним ударом

`Clamp()` удерживает значения в рабочем диапазоне. Профиль минимальный: броня, слабые точки, стихийные сопротивления и скриптовая неуязвимость — это игровые правила, они живут в пользовательской политике или в таблице материалов игры.

5.2 Части и связи

Модульный объект — это `SdfDamageModel` с максимум 32 частями и 64 связями.

```
Fby::SdfDamageModel Damage;

Fby::SdfPart Shell;
Shell.Center = Fby::Vector2D{0.0f, 0.0f};
Shell.Radius = 24.0f;
Shell.Integrity = 1.0f;
Shell.Mass = 4.0f;
Shell.Material = 0;
Damage.AddPart(Shell);

Fby::SdfPart Core;
Core.Center = Fby::Vector2D{0.0f, 0.0f};
Core.Radius = 10.0f;
Core.Integrity = 1.0f;
Core.Mass = 8.0f;
Core.Material = 1;
Damage.AddPart(Core);

Fby::SdfConnection Link;
Link.PartA = 0;
Link.PartB = 1;
Link.Strength = 0.8f;
Damage.AddConnection(Link);
```

`SdfPart::Integrity` нормирована в 0..1. `SdfPart::Radius` — дешёвый прокси для попадания по точке; настоящую форму фрагмента игра может хранить в своём ассете. `SdfConnection` — структурный граф: сломанная часть рвёт все связи, которые её касаются. `ApplyConnectionLoad` накапливает внешнюю нагрузку и рвёт

связь, когда Load достигает Strength; это простая точка, к которой игра может подключить собственное распространение напряжений.

5.3 Удар

SdfImpact описывает разрешённое попадание:

Поле	Значение
Target	Сущность, по которой попали
Point	Мировая точка контакта
Normal	Внешняя нормаль
RelativeSpeed	Скорость сближения
Impulse	Корректирующий импульс
KineticEnergy	Энергия удара до коррекции
Mass	Масса тела
LocalThickness	Глубина материала по внутренней нормали; ноль означает «неизвестно», правило по умолчанию берёт единицу

MakeSdfImpact(contact, field, maxThickness) копирует контакт и оценивает локальную толщину. SdfThicknessInward(field, point, normal, maxThickness) движется от поверхности вглубь материала и возвращает ноль для точки, уже находящейся снаружи.

```
for (const Fby::SdfContact& Contact : Collision.Contacts())
{
    const Fby::SdfImpact Impact = Fby::MakeSdfImpact(Contact, Field, 24.0f);

    const std::uint32_t PartIndex = Damage.FindPartAtPoint(Impact.Point);
    if (PartIndex == Fby::kSdfInvalidDamagePart)
    {
        continue;
    }

    const Fby::SdfPart* Part = Damage.Part(PartIndex);
    const Fby::SdfMaterialProfile& Material = Materials[Part->Material];
    const Fby::SdfImpactResult Result = Damage.ApplyImpact(Impact, PartIndex,
Material);

    if (Result.PartBroken)
    {
        // На стороне игры: авторские фрагменты, осколки, звук и VFX.
    }
}
```

ApplyImpactAtPoint — удобный вариант для одного профиля материала:

```
const Fby::SdfImpactResult Result = Damage.ApplyImpactAtPoint(Impact, Material);
```

5.4 Правило по умолчанию и пользовательская политика

`SdfDefaultDamage` вычисляет:

```
damage = clamp(
    (KineticEnergy - MinKineticEnergy) * EnergyToDamage * Hardness
    / (Toughness * LocalThickness^ThicknessExponent),
    0,
    MaximumDamageFraction)
```

Результат — доля целостности части. Пользовательская политика заменяет формулу:

```
float ArmourPolicy(void* User, const Fby::SdfImpact& Impact,
                   const Fby::SdfPart& Part,
                   const Fby::SdfMaterialProfile& Material) noexcept
{
    const auto* Layers = static_cast<const ArmourLayers*>(User);
    // Игровое правило: направление, слабая точка, стихийное сопротивление.
    return ComputeArmourDamage(*Layers, Impact, Part, Material);
}
const Fby::SdfImpactResult Result =
    Damage.ApplyImpact(Impact, PartIndex, Material, &ArmourPolicy, &Layers);
```

Политика должна возвращать 0..1, не использовать случайность вне

записываемого состояния симуляции и не читать системные часы.

5.5 Передача в разлом и осколки

Когда `Integrity` достигает нуля:

- `SdfImpactResult::PartBroken` равен true;
- `SdfImpactResult::BrokenConnections` содержит число связей, сменивших

состояние;

- `SdfPart::Broken()` и `SdfConnection::Broken()` становятся true;
- повторные удары по этой части игнорируются.

Движок не решает, чем становится сломанная часть. Игра может:

1. вызвать `FractureGrid` для прототипа;
2. создать сущности авторских фрагментов через `SpawnDebris` или свой ассет;
3. удалить объект и впечатать поле через `SdfWorld::Imprint`;
4. проиграть звук, VFX, выдать лут и обработать победу или поражение.

`FractureGrid` копирует полный список операций в каждый кусок. Это корректно для

маленьких прототипов, но дорого для сложной структуры; production-путь —

авторские фрагменты.

6. Детерминизм, сохранение и сеть

- `SdfDamageModel` использует фиксированные массивы, не имеет RNG и не выделяет память. Одинаковые входы дают одинаковое состояние.
- Пользовательская политика урона становится частью контракта симуляции. Если реплей не может выполнить ту же политику, он должен явно упасть, а не разойтись.
- Контакты `SdfCollisionSystem` не сериализуются. Сериализуется урон: игра должна хранить целостность частей, флаги сломанных частей и связей либо воспроизводить команды урона в том же порядке.
- `SdfDestructionCommands` уже записывает операции вырезания. Будущая команда урона должна записывать удар или итоговый урон, но не оба сразу.
- Нельзя подавать GPU-readback или состояние партиклов рендера в урон. Авторитетное состояние остаётся на CPU.
- Новые сериализуемые поля требуют повышения версии формата и теста миграции.

Текущие структуры `SdfDamage` существуют только в рантайме; раскладка `.fsdf` не менялась.

7. Чего движок не делает

- Универсальной континуальной/FEM-деформации. Она дорогая, сложно настраивается и обычно не нужна 2D-игре на спрайтах.
- Автоматического переноса текстуры на вырезанную или разломанную форму. Используйте маску повреждения, слой внутреннего среза, краевые декали и авторский атлас фрагментов.
- Автоматической деформации коллайдера из косметического warp спрайта. Визуальная вмятина может использовать warp меша, но авторитетный коллайдер остаётся выпуклой формой или SDF-полем.
- Универсального лута, счёта, победы/поражения или правила кислорода и

пузырей. Это собственность игры.

- Собственного арта фрактур. Движок даёт состояние урона и существующие

примитивы фрактур/осколков; арт-пайплайн даёт куски.

8. Рекомендуемые уровни 2D-разрушения

Уровень	Авторинг	Рантайм	Применение
0	Один спрайт, спрайты повреждений	Смена спрайта, декали	Дешёвый декор
1	Авторские выпуклые фрагменты	Разрыв связей, спавн кусков	Основное геймплейное разрушение
2	Меш спрайта с контрольными точками	Косметический warp без смены коллайдера	Вмятины и изгибы
3	Локальное SDF-поле	`SdfCarveOp`, imprint	Пулевые отверстия и детали
4	Внешний решатель	Игра или плагин	Настоящая мягкая материя

Уровень 1 — рекомендуемый по умолчанию. Уровень 3 — для локальных деталей, а не

для структурного графа. Уровень 4 не должен обещаться универсальным 2D-движком.

9. Бюджеты и тесты

- `SdfDamageModel`: 32 части, 64 связи, один проход по частям для поиска точки,

один проход по связям для разрыва.

- `SdfCollisionSystem`: одна билинейная выборка и четыре выборки градиента на

тело за фиксированный шаг, число контактов ограничено `kMaxContacts`.

- `SdfDestructible`: один локальный перепёк блоков на вырезание, не более 64

операций.

- Работа с уроном от контакта — O(частей) на удар, без выделения памяти.
- Модель урона не потокобезопасна. Выполняйте её в потоке симуляции, после шага

SDF-коллизий, до фрактуры.

Тесты:

```
ctest --test-dir build-dev -R "sdf_(physics|damage|destructible|fracture|debris)" --
output-on-failure
```

- `sdf_physics`: выборка SDF, контакты, манифолды и новые поля удара до

коррекции.

- `sdf_damage`: ограничение материала, части, связи, поиск по точке, правило по умолчанию, пользовательская политика, толщина и распространение разрыва.
- `sdf_destructible`, `sdf_fracture`, `sdf_debris`: геометрия и примитивы фрагментов, которые питает модель урона.

10. Краткий справочник API

Тип или функция	Назначение
`SdfContact`	Разрешённый SDF-контакт с полями удара
`SdfMaterialProfile`	Физические данные материала
`SdfPart`	Модульная часть с целостностью и id материала
`SdfConnection`	Структурная связь двух частей
`SdfImpact`	Разрешённый удар, готовый для правила урона
`SdfImpactResult`	Урон, разлом и результат по связям
`SdfThicknessInward`	Локальная глубина материала по нормали
`MakeSdfImpact`	Преобразование контакта в удар
`SdfDamageModel`	Хранилище частей и связей фиксированной ёмкости
`ApplyConnectionLoad`	Добавляет структурную нагрузку и рвёт связь на `Strength`
`SdfDefaultDamage`	Нейтральное правило по умолчанию
`SdfDamagePolicy`	Колбэк пользовательского правила
`PhysicsSystem::SetFixedRotation`	Блокирует или освобождает угловую свободу тела
`PhysicsSystem::SetBullet`	Включает или выключает непрерывное обнаружение коллизий
`TransportGroupSystem`	Создаёт, вкладывает, переносит, отцепляет и обновляет транспортные группы
`TransportGroupPanel`	Панель редактора для корней групп, членов и политик

11. Транспортные группы: лифты, поезда и платформы

Транспортная группа — это кинематический носитель, а не иерархия твёрдых тел. Она решает случай, когда платформа едет под персонажем или поезд везёт ящики, которые при этом должны иметь возможность двигаться самостоятельно.

Не строите это на `Parent2D`: этот компонент складывает позицию, масштаб и поворот в мировой трансформ и предназначен для визуала. Динамический ящик начал бы наследовать поворот и масштаб поезда. Транспортная группа — отдельный граф трансформов.

Компоненты

Компонент	Поля	Назначение
`TransportGroupComponent`	`GroupId`, `ParentGroupId`, `RotationPolicy`, `Flags`, `BoundsMin`, `BoundsMax`, `ParentJoint`	Ставится на корень группы
`CarrierMemberComponent`	`GroupId`, `Mode`, `Flags`, `LocalPosition`, `LocalRotation`, `RelativeVelocity`, `RelativeAngularVelocity`, `Joint`	Ставится на переносимую сущность
`TransportPath`	`Points`, `Count`, `Closed`	Опциональная полилиния для кинематического корня
`TransportMotion`	`Speed`, `Acceleration`, `Deceleration`, `DwellSeconds`, `Distance`, `Direction`, `Axis`, `Flags`	Кинематическое движение по пути или по оси; ramp-поля зарезервированы для контроллера
`TransportSurface`	`Friction`, `TangentSpeed`, `OneWay`, `AllowJump`	Политика опоры и конвейера

Все транспортные компоненты зарегистрированы и видны в инспекторе. Id группы — стабильное целое число, а не runtime-хэндл. `ParentJoint` и `Joint` — runtime-идентификаторы Box2D: не задавайте их вручную, система проверяет и пересоздаёт их после загрузки сцены. `TransportPath` отдаёт в reflection только `Count` и `Closed`; массив точек задаётся кодом или проектным ассет-инструментом. Motion и surface не входят в rollback-схему `SnapshotSchema::BuiltIns`; сетевой игре нужно явно реплицировать или пересоздавать их.

Режимы членства

Режим	Поведение
`kCarrierModeRigid`	Точное крепление; при наличии обоих тел используется weld-джойнт Box2D, иначе точный локальный трансформ
`kCarrierModeCarriedDynamic`	Тело остаётся динамическим; носитель добавляет свою скорость, а тело сохраняет собственное относительное движение
`kCarrierModeSupport`	Контактная опора: нормальная поддержка, трение и runtime-бит `grounded`
`kCarrierModeConveyor`	Тангенциальная скорость поверхности для лент и движущихся дорожек

`kCarrierFlagInheritRotation` определяет, следует ли член за поворотом носителя. Персонажу обычно нужно сохранять вертикаль; ящик может наследовать поворот.

Движение, опора и DOD

`TransportMotion` обновляется до вычисления дельты группы. Путь сэмплируется по длине дуги; `Closed`, ping-pong, остановка на концах и поворот по направлению задаются флагами. Без пути корень движется по `Axis`. Корень с `RigidBody2D` телепортируется, чтобы солвер увидел позу; корень только с `Transform2D` перемещается напрямую.

`Support` и `Conveyor` используют `TransportSurface`, а не слепое копирование скорости. `Support` сохраняет относительное движение и применяет трение к поверхности; `Conveyor` ведёт тангенциальную скорость к `TangentSpeed`.

Геометрическая проверка опоры читает AABB носителя и члена и ставит `kCarrierFlagGrounded`, поэтому правило одинаково работает в Box2D и DOD.

В DOD нет joints, weld и CCD. `TransportGroupSystem::CapabilitiesFor()` сообщает об этом; `Rigid` откатывается к телепорту локальной позы, а сцепленным поездам нужен Box2D. Движение фиксированного шага и основано на расстоянии, поэтому детерминировано

при одинаковом начальном состоянии. `TransportPath`, `TransportMotion` и `TransportSurface` не входят в `SnapshotSchema::BuiltIns`; для rollback или сети их нужно пересоздавать или реплицировать. Runtime-хэндлы соединений и состояние группы транзиентны.

Политика вращения

`RotationPolicy` — метаданные для игры и редактора:

- `kTransportRotationFixed`: угловая степень свободы корня блокируется через `PhysicsSystem::SetFixedRotation`; корень может поступательно двигаться и быть динамическим телом.
- `kTransportRotationAuthored`: поворотом корня управляет игра или путь; фиксация вращения снимается.
- `kTransportRotationPhysics`: свободное вращение физикой, включая момент и коллизии. Это явный полностью физический уровень.

Система никогда не добавляет момент корню. `Fixed` блокирует вращение, `Authored` отдаёт его пути, `Physics` — солверу.

Запуск системы

```
Fby::TransportGroupSystem Groups;
Groups.CreateGroup(Registry, TrainRoot, 1);
Groups.Attach(Registry, Crate, 1, Fby::kCarrierModeRigid);
Groups.Attach(Registry, Character, 1, Fby::kCarrierModeSupport);

for (const Tick& ScopedTick : Ticks)
{
    Groups.PrePhysics(Registry, Physics, ScopedTick.Dt);
    Physics.Step(Registry, ScopedTick.Dt);
    Groups.PostPhysics(Registry, Physics, ScopedTick.Dt);
}
```

`PrePhysics` считает дельты групп, применяет движение родителя к вложенным корням и передаёт скорость носителя динамическим членам. `PostPhysics` обновляет локальные позы, относительные скорости и условия отцепления. Порядок важен: шаг физики должен находиться между этими двумя вызовами.

Вложенные группы

Поезд может содержать подгруппу лифта:

```
Groups.CreateGroup(Registry, TrainRoot, 1);
Groups.CreateGroup(Registry, LiftRoot, 2, 1);
Groups.SetParentGroup(Registry, 2, 1);
```

Вложенные корни получают дельту родителя один раз. Если у обоих корней есть физические тела, дочерний корень несёт родительский weld-джойнт; иначе применяется дельта трансформа. Член, прикреплённый к лифту, получает суммарную мировую дельту лифта, а не дельту поезда дважды. Родительские id проверяются на циклы.

Прикрепление и отцепление

`Attach` сохраняет локальную позу члена и, для динамического тела, скорость относительно носителя. `Detach` убирает член:

- `CarriedDynamic` сохраняет текущую скорость тела, поэтому инерция уже сохранена;
- `Rigid` уничтожает свой weld-джойнт и оставляет телу скорость от солвера;

rigid-член без тела сохраняет текущий трансформ;

- `Detach(..., false)` обнуляет скорость тела для явного сброса.

Член может запросить отцепление флагом `kCarrierFlagDetachRequested`. Группа с `kTransportGroupAutoDetachOutside` также отцепляет членов, чья локальная точка вышла за `BoundsMin`/`BoundsMax`. Условие задаёт игра; система только выполняет его.

Сборка в редакторе

В меню View есть отдельная панель `Transport Groups`. Она создаёт группу на выбранной сущности, редактирует id группы, id родителя, политику вращения, флаги enabled/auto-detach и bounds, прикрепляет выбранную сущность как члена, показывает список членов, отцепляет их и удаляет выбранную группу. Для выбранного корня она позволяет добавить и редактировать `TransportMotion` и `TransportSurface`; для колеса или шасси показывает поля привода, подвески и руля `Wheel2D`/`Vehicle2D`. Данные остаются обычными зарегистрированными компонентами, поэтому видны в общем инспекторе и следуют политике сохранения компонентов проекта.

Доступ из скрипта

Скриптовый рантайм получает сервис групп через `ScriptRuntime::SetTransportGroups`. Нативные функции:

Нативная функция	Аргументы
`transport_attach`	id группы, режим (прикрепляет текущую сущность)
`transport_detach`	Без аргументов (отцепляет текущую сущность)
`transport_set_parent`	id группы, id родительской группы
`transport_create`	id группы, id родителя, политика вращения (создаёт группу на текущей сущности)

Компоненты также зарегистрированы в ECS bridge, поэтому граф может выставить `CarrierMemberComponent::Flags` и запросить отцепление без нативного вызова.

Динамические корни и большие скорости

- Корень может быть кинематическим или динамическим. `Fixed` блокирует только

угловую степень свободы, поэтому лифт может быть динамическим телом, которое едет, но не опрокидывается. `Physics` оставляет полное вращение солверу.

- Переносимое тело остаётся обычным телом Box2D: коллизии, гравитация и демпфирование не меняются.
- В Box2D носители и переносимые тела автоматически включают CCD, когда скорость

превышает `kTransportHighSpeedPixelsPerSecond`. Фиксированные подшаги всё ещё остаются выбором автора для экстремальных скоростей; автоматический путь — страховка, а не замена разумному фиксированному шагу.

- `Rigid`-член, наследующий поворот и имеющий оба тела, держится weld-джойнтом, а не телепортом каждый шаг. Fallback без наследования поворота и без тела использует точный локальный трансформ. Это снимает конфликт телепорта и коллизий для обычного жёсткого груза.
- Вложенные корни, у которых есть тела, несёт родительский weld-джойнт.

Вложенные корни без тел используют путь дельты родителя. Поддерживаются оба варианта.

12. Колёса транспорта

`WheelSystem` — небольшая явная база транспорта на wheel-джойнтах Box2D. Она отвечает за подвеску, привод и торможение каждого колеса. Она не моделирует RPM двигателя, коробку, сцепление, дифференциал, проскальзывание и

износ шин или геометрию рулевого управления.

Компоненты

Компонент	Поля	Назначение
`Vehicle2D`	`Throttle`, `Brake`, `Steering`, `MaxMotorTorque`, `MaxSpeed`, `SteeringTorque`, `Flags`	Ввод и ограничения шасси
`Wheel2D`	`Chassis`, `LocalAnchor`, `SuspensionAxis`, `Radius`, `SuspensionHertz`, `SuspensionDamping`, `DriveWeight`, `BrakeWeight`, `Flags`, `Joint`	Одно колесо

Колесо — обычное динамическое тело `RigidBody2D` с `Collider2D`. Система не создаёт и не расставляет колёса автоматически: это авторское решение шасси.

`Joint` — runtime-идентификатор wheel-джойнта Box2D, его нельзя сериализовать.

Порядок

```
WheelSystem::Initialize(physics)
WheelSystem::Update(registry, physics, fixedDt)
PhysicsSystem::Step(registry, fixedDt)
```

`Update` лениво создаёт недостающие джойнты и задаёт пружину, скорость и момент мотора до шага солвера. Линейную скорость тела он не перезаписывает.

Привод, тормоз и руль

- `Throttle` задаёт угловую скорость `throttle * MaxSpeed / Radius`.
- Момент на колесо: `abs(throttle) * MaxMotorTorque * DriveWeight`.
- `DriveWeight = 0` — свободное колесо; передний/задний/полный привод задаётся по колёсам, а не угадывается движком.
- `Brake` выключает мотор колеса и применяет тормозной момент плюс явное угловое демпфирование, чтобы свободно крутящееся колесо останавливалось.
- `VehicleFlagHandbrake` тормозит все колёса с ненулевым `BrakeWeight`.
- `VehicleFlagArcadeSteering` включается явно. Только колёса с `kWheelFlagSteer` применяют скорость-зависимый момент рыскания шасси.

Физическая геометрия руля — решение игры/плагина, потому что wheel-джойнт Box2D не даёт задать угол поворота.

- Движок не добавляет дифференциал и автоматическую заднюю передачу.

Бэкенд

В DOD нет wheel-джойнтов. `WheelSystem::Initialize` возвращает `false`, а `Available()` сообщает об ограничении. Колёсному транспорту нужен Box2D.

Граница authored/game описана в [vehicle wheels](../../design/vehicle-wheels.md).